

ASTRALIS

Documento de Arquitectura

Cristian Moreno

Stephan Garcia

Santiago Zamora

Daniel Garzon

Ingeniería de Software II · UNAL 2026-1S

Observatorio Astronómico Nacional

Roles del equipo

Cristian Moreno Backend	Stephan Garcia DevOps	Santiago Zamora ML & Física	Daniel Garzon Frontend
FastAPI · PostgreSQL · JWT	GitHub Actions · Docker	scikit-learn · rebound · NASA APIs	React · Three.js · Vite

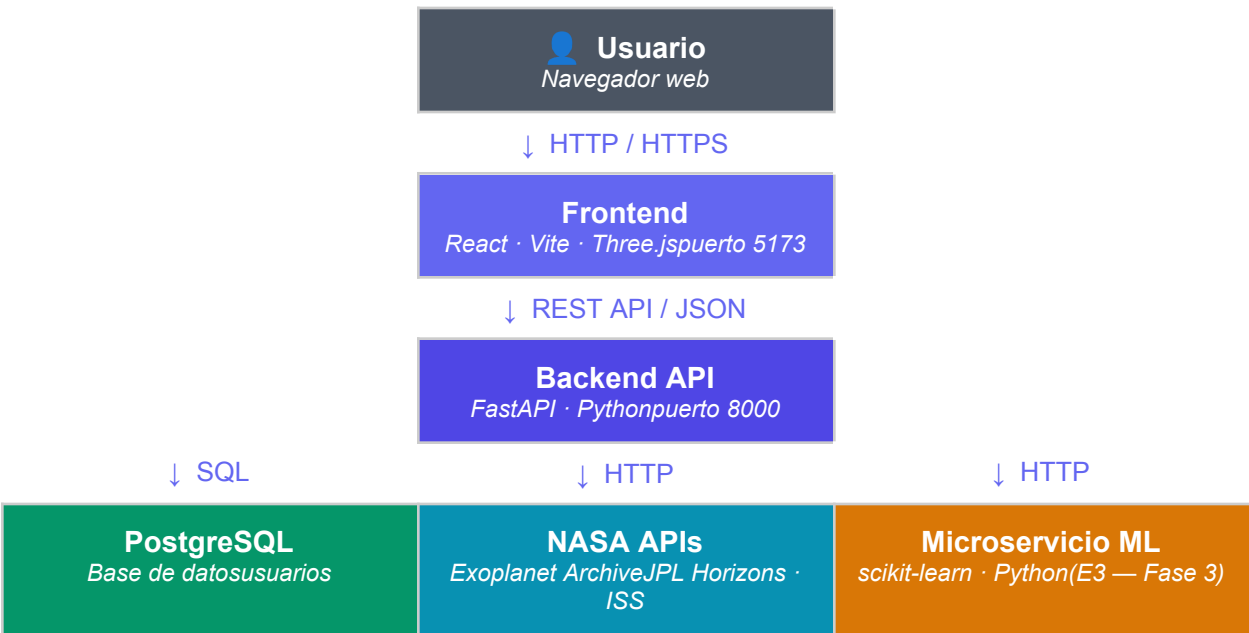
1. Descripción General del Sistema

ASTRALIS es una plataforma web que permite simular, visualizar y predecir fenómenos astronómicos usando datos reales de la NASA. El sistema integra cuatro módulos técnicos desarrollados por roles independientes que se comunican a través de una API REST central.

El sistema está diseñado con las mismas preocupaciones de cualquier producto profesional en producción: escalabilidad, seguridad, automatización y ética de software.

2. Diagrama C4 — Nivel 2 (Contenedores)

El diagrama muestra los contenedores que forman el sistema ASTRALIS y cómo se comunican entre sí. Cada contenedor es una unidad desplegable de forma independiente.



2.1 Descripción de contenedores

Contenedor	Tecnología	Puerto	Responsable
Frontend SPA	React + Vite	5173	E4
Backend API REST	FastAPI + Python	8000	E1
Base de datos	PostgreSQL 18	5432	E1
Microservicio ML	scikit-learn + FastAPI	8001	E3

3. Diseño de la API REST

3.1 Endpoints implementados

Método	Ruta	Descripción
POST	/api/auth/register	Registro de usuario nuevo
POST	/api/auth/login	Login → devuelve JWT
GET	/api/exoplanets	Lista paginada con filtros por clasificación
GET	/api/exoplanets/{id}	Detalle de un exoplaneta
GET	/api/exoplanets/{id}/lightcurve	Curva de luz de tránsito del exoplaneta
GET	/api/iss/position	Posición actual de la ISS en tiempo real
GET	/api/simulation/bodies	Cuerpos del sistema solar para el simulador
GET	/health	Health check para el pipeline CI/CD de E2

3.2 Autenticación

El sistema usa JWT (JSON Web Tokens) con algoritmo HS256. El token se incluye en el header Authorization: Bearer <token> en cada request protegido. Los tokens tienen expiración configurable (default: 60 minutos).

3.3 Modelo de datos — Entidad Relación

Entidad users	Entidad exoplanets*	Entidad simulations (Fase 3)
id (UUID, PK)username (String)email (String, único)hashed_password (String)	id (kepid, String, PK)name · hostStarorbitalPeriod · planetRadiusequilibriumTempclassification · mlConfidencediscoveryYear	id (UUID, PK)user_id (FK → users)config (JSONB)result (JSONB)created_at

** Los exoplanetas se cargan desde el CSV del NASA Exoplanet Archive. No se almacenan en PostgreSQL para evitar duplicar datos que ya mantiene la NASA.*

4. Fuentes de Datos

Fuente	Datos	Usado por
--------	-------	-----------

NASA Exoplanet Archive	~9.000 candidatos Kepler con koi_disposition, koi_score, parámetros orbitales	Backend (E1) · Modelo ML (E3)
Open Notify / wheretheiss.at	Posición ISS en tiempo real (lat, lon, alt, velocidad)	Backend (E1) · Frontend (E4)
JPL Horizons System	Posiciones y velocidades de cuerpos del sistema solar	Backend (E1) · Simulador (E3)

5. Architecture Decision Records (ADR)

Los ADR documentan las decisiones técnicas importantes del proyecto y la justificación detrás de cada una. Sirven como referencia para el equipo y para quien se incorpore al proyecto en el futuro.

ADR-001 · FastAPI como framework del backend

Campo	Descripción
Estado	Aceptado
Contexto	El equipo necesitaba un framework Python para construir la API REST que sirve datos de NASA, autenticación JWT y actúa como proxy hacia APIs externas.
Decisión	Se usa FastAPI como framework principal del backend.
Razones	1. Generación automática de documentación Swagger/OpenAPI sin configuración adicional, requerida explícitamente en Fase 3. 2. Soporte nativo para operaciones async/await, necesario para múltiples llamadas paralelas a APIs externas (NASA, ISS, JPL). 3. Validación automática de request/response con Pydantic. 4. Alto rendimiento comparable a Node.js según benchmarks de TechEmpower.
Alternativas descartadas	Django REST Framework: mayor overhead, pensado para aplicaciones más grandes con ORM propio. Flask: más flexible pero requiere más configuración manual para validación y documentación.
Consecuencias	El equipo debe aprender Pydantic para definir schemas. Los modelos de request/response se definen dos veces: en TypeScript (frontend) y en Pydantic (backend). Esto es intencional — garantiza contratos explícitos.

ADR-002 · JWT para autenticación (sin sesiones de servidor)

Campo	Descripción
Estado	Aceptado
Contexto	El sistema necesita autenticación para proteger endpoints. La arquitectura es frontend SPA + backend API desacoplados.
Decisión	Se usa autenticación basada en JWT (JSON Web Tokens) con algoritmo HS256.
Razones	1. Stateless: el servidor no almacena sesiones, facilitando el escalado horizontal en Railway/Render. 2. Compatible con arquitecturas SPA donde el frontend y backend están en dominios distintos.

	3. El token viaja en el header Authorization, compatible con el interceptor de Axios ya implementado en el frontend. 4. Estándar de la industria para APIs REST públicas. 5. Permite que el microservicio ML de E3 también valide tokens sin compartir estado con el backend principal.
Alternativas descartadas	Sesiones con cookies: requieren mismo dominio o configuración CORS compleja. OAuth2 completo con proveedor externo: overhead innecesario para un sistema académico sin usuarios externos.
Consecuencias	Los tokens no se pueden revocar antes de su expiración sin implementar una blacklist. Para ASTRALIS esto es aceptable dado el contexto académico. En producción real se implementaría refresh tokens con rotación.

ADR-003 · Monorepo con frontend y backend en carpetas separadas

Campo	Descripción
Estado	Aceptado
Contexto	El proyecto tiene 4 roles con responsabilidades técnicas distintas. E2 necesita pipelines CI/CD independientes por módulo. E1 y E4 trabajan en stacks completamente diferentes (Python vs TypeScript).
Decisión	Se usa un monorepo único con carpetas separadas: frontend/ para React y backend/ para FastAPI. El microservicio ML de E3 vivirá en science/ en Fase 3.
Razones	1. Un solo repositorio facilita la coordinación del equipo — un PR puede tocar frontend y backend al mismo tiempo si un cambio lo requiere. 2. E2 puede configurar un pipeline CI/CD unificado con jobs separados por carpeta usando paths en GitHub Actions. 3. Visibilidad compartida, todos los roles ven el estado del sistema completo en un solo lugar. 4. Historial unificado, los cambios de contrato entre frontend y backend quedan en el mismo commit. 5. Más simple para un equipo de 4 personas.
Alternativas descartadas	Repositorios separados por rol: complica la coordinación cuando un cambio de API requiere actualizar frontend y backend simultáneamente. Requiere sincronizar PRs entre repos distintos.
Consecuencias	E2 debe configurar los workflows de GitHub Actions con filtros paths para que el pipeline de backend solo corra cuando cambia backend/ y viceversa. El contrato entre frontend y backend se documenta en Swagger y se revisa en cada PR.

6. Stack Tecnológico

Capa	Tecnologías	Justificación
Backend	FastAPI · PostgreSQL · JWT · bcrypt	Ver ADR-001 y ADR-002
Frontend	React · TypeScript · Vite · Three.js · Recharts	SPA moderna, Three.js para visualización 3D orbital
DevOps	GitHub Actions · Docker	Gratuitos, usados en la industria colombiana
ML & Física	scikit-learn · rebound · Jupyter	Dataset Kepler + simulación N-cuerpos
Calidad	pytest · ESLint · OWASP ZAP	Cobertura mínima 60% requerida en Fase 2
Monitoreo	UptimeRobot · Grafana Cloud	Health check en /health para alertas

7. Decisiones Fuera del Alcance

Las siguientes decisiones ya están tomadas y no serán reconsideradas durante el semestre:

- Procesamiento de datos LIGO / ondas gravitacionales
- GraphQL — REST es suficiente para todos los casos de uso del sistema
- Aplicación móvil nativa — la interfaz web es responsive
- Visualización 3D avanzada con física compleja — es un bonus opcional de Fase 3